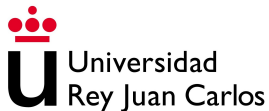


Tema 1: Especificación formal

Metodología de la Programación



Objetivos

Objetivos

- Revisar los principales conceptos de la lógica de primer orden
- Entender el concepto de estado de cómputo y cómo se modela con predicados lógicos
- Familiarizarse con los elementos de la especificación formal
- Especificar formalmente algoritmos simples

Índice

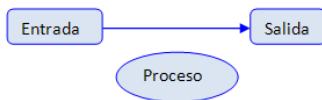
- 1 Introducción
- 2 Lógica de proposiciones
- 3 Lógica de predicados
- 4 Especificación con predicados
- 5 Especificación pre/post

Introducción

Introducción

↪ Algoritmos y programas

- Un **algoritmo** es la descripción precisa de los pasos que nos llevan a la solución de un problema planteado.
- La descripción de un algoritmo afecta a:



- Un **programa** es la codificación en un lenguaje de programación concreto de un algoritmo.
- Cuando se diseña un algoritmo:
 - Primero se **especifica** y luego se **implementa**.

Introducción

↪ Especificación e implementación

- **Especificar**, contestar a la pregunta *¿Qué hace el algoritmo?*, sin dar ningún detalle de cómo se hace
- **Implementar**, contestar a la pregunta *¿cómo se consigue el algoritmo requerido?*

↪ Especificación

- Consiste en detallar cuidadosamente el problema que se quiere resolver.
- **Destino** de la especificación de un algoritmo:
 - *Los usuarios*: la especificación debe recoger todo lo necesario para un uso correcto del algoritmo $\equiv$ obligaciones del usuario al invocar el algoritmo
 - *El implementador*: la especificación define los requisitos que cualquier implementación válida debe satisfacer $\equiv$ obligaciones del implementador.
- **Especificación precisa**: Debe responder a cualquier pregunta sobre el uso del algoritmo sin tener que acudir a la implementación. *¿Para qué sirve?* o *¿cómo se utiliza?*.
- Debe ser clara y precisa a la vez.
 - El lenguaje natural no permite la precisión requerida si no es sacrificando otra propiedad que ha de tener toda buena especificación: **la brevedad**.

Introducción

Ejemplo

Queremos una función que, dado un vector $V[1..1000]$ de *enteros* y un entero n , devuelva un valor booleano que indique si el valor de alguno de sus elementos $V[1], V[2], \dots, V[n]$ es igual a la suma de todos los elementos que le preceden en el vector.

fun essuma($V[1..1000]$ de enteros, n : entero) dev (b : booleano)

→ **Especificación:** "Una función que dado un vector de mil posiciones devuelva verdadero si alguno de sus elementos es igual a la suma de todos los elementos que la preceden en el vector".

- ¿obligaciones del usuario? $\Rightarrow$ no quedan claras
 - ¿serían admisibles llamadas a "essuma" con valores negativos de n ?
 - ¿Y con $n=0$?
 - ¿Y con $n > 1000$?
- ¿obligaciones del implementador? $\Rightarrow$ no quedan claras
 - Si $n \geq 1$ y $v[1] = 0 \rightarrow$ ¿la función ha de devolver *cierto* o *falso*?

→ Para que las **especificaciones sean precisas y claras** se requiere una **especificación formal**.

Lógica de proposiciones

Lógica de proposiciones

- Simboliza y describe razonamientos basados en enunciados declarativos. Por ejemplo:
 - *Está lloviendo*
 - *Antonio corre*
 - $3+2=5$
- Una proposición es un enunciado declarativo que puede ser verdadero o falso, pero no ambos a la vez.
 - "*levántate y anda*" no es una proposición.
- La extiende la lógica de predicados, que permite hablar de objetos y relaciones entre ellos.
 - *Algunas plantas tienen flores*
 - *Todos los hombres son mortales*

Lógica de proposiciones

- Proposiciones

- Son los enunciados: "*Está lloviendo*", "*Antonio corre*", " $3+2=5$ ".
- Se representan mediante variables proposicionales simbolizadas mediante letras: p, q, r, s, t, etc.
- Sólo pueden ser verdaderas o falsas

- Fórmulas sentenciales

- Combinación de variables proposicionales y conjunciones (y,o,...).
- El valor de verdad dependerá del valor de las variables (V/F) y de las conjunciones

Lógica de proposiciones - Sintaxis

↪ Alfabeto

- Los símbolos de verdadero o falso (V/F).
- Los símbolos que representan variables proposicionales ($p, q, r, s, \dots$).
- Los símbolos que representan conectivas:
 - Conjunción (y): $\wedge$
 - Disyunción(inclusiva)(o): $\vee$
 - Disyunción exclusiva (o..o): $\oplus$. Una u otra pero no ambas a la vez
 - Negación (no): $\neg$
 - Condicional (si...entonces...): $\rightarrow$
 - Bicondicional (si y sólo si): $\leftrightarrow$
 - Los símbolos (,) para evitar ambigüedades.
- Reglas de formación
 - **Fórmula sentencial**: secuencia finita de símbolos de variables proposicionales, conectivas y paréntesis.

$$) \vee p \rightarrow ($$

$$p \wedge (q \vee \neg r) \rightarrow s$$
 - **Fórmula bien formada**: aquella fórmula sentencial que tiene sentido.

Lógica de proposiciones - Sintaxis

- Las tres **reglas de formación**, de sentencias bien formadas, son:
 - 1 Una variable proposicional es una sentencia bien formada.
 - 2 Una sentencia bien formada precedida de la negación ($\neg$) es una sentencia bien formada.
 - 3 Dos sentencias bien formadas S y R unidas por una de las partículas conectivas binarias $(S \wedge R), (S \vee R), (S \rightarrow R), (S \leftrightarrow R), (S \oplus R)$, constituyen una sentencia bien formada.
- Ejemplos:
 - ▷ p
 - ▷ $\neg p$
 - ▷ $\neg(p \wedge q)$
 - ▷ $[\neg p \vee (q \leftrightarrow p)]$

Lógica de proposiciones - Semántica

- Asigna a cada sentencia de la lógica proposicional un **valor de verdad** V/F
- La asignación de unos valores concretos, a cada variable proposicional, corresponde a una **interpretación**.

S: $p \rightarrow (p \vee q)$

- **Tablas de verdad**
 - Permite determinar el valor de verdad de una sentencia para una interpretación dada.
 - Tabla donde se presentan **todas** las posibles interpretaciones de las variables proposicionales y el valor de verdad de cada interpretación.
 - Modo mecánico de determinar la verdad o falsedad de una sentencia dada una interpretación de las variables

Lógica de proposiciones - Semántica

- Tablas de verdad para las conectivas $\neg, \wedge, \vee, \oplus, \rightarrow, \leftrightarrow$.

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \oplus q$	$p \rightarrow q$	$p \leftrightarrow q$
V	V	F	V	V	F	V	V
V	F	F	F	V	V	F	F
F	V	V	F	V	V	V	F
F	F	V	F	F	F	V	V

- $\oplus$: Verdadera cuando sólo una variable proposicional sea verdadera; falso en caso contrario.
- $\rightarrow$: Falsa cuando p sea verdadera y q sea falsa; verdadera en caso contrario.
- $\leftrightarrow$: Verdadera cuando las dos variables proposicionales sean verdaderas o las dos falsas.
- Tautologías** o sentencias válidas: sentencias que siempre son verdaderas para cualquier interpretación (tabla de verdad siempre V)

$$p \vee (\neg p)$$
- Contradicciones**: sentencias que siempre son falsas para cualquier interpretación (tabla de verdad siempre F)

$$p \wedge (\neg p)$$

Lógica de predicados

Lógica de predicados

- Hay situaciones que son imposibles de expresar en lógica de proposiciones.

Todos los hombres son mortales

Sócrates es un hombre

Luego Sócrates es mortal

- Hablamos:

- Propiedades de individuos (ser hombre, ser mortal)
- Se pueden aplicar a un individuo cualquiera o a uno concreto (variable o constante)

- Se necesitan cuantificadores para expresar *para todos los individuos* o para *uno* en concreto, etc.

→ Lógica de predicados

- Extiende la lógica de proposiciones y permite hablar de objetos y relaciones entre ellos.

Lógica de predicados - Sintaxis

- Las letras mayúsculas P, Q, R designan predicados
- Las letras minúsculas $a, b, \dots, x, \alpha, \beta, \dots$, designan variables

↪ Predicados

Una *fórmula* en lógica de predicados o más brevemente, un *predicado*, es:

1 Una *fórmula atómica*

- Las constantes booleanas, verdadero y falso
- Una variable booleana
- Las expresiones aritméticas relacionales tales como:
 $2 * x = y$ $u \bmod v \neq 0$ $a > 0$ $i \leq j$
- Una función matemática conocida o especificada formalmente, que devuelva un resultado booleano, como:
 $\text{primo}(x)$ $\text{divide}(x, y)$

2 Una negación, $\neg P$, siendo P un predicado

3 Una conjunción, $P \wedge Q$, siendo P y Q predicados.

4 Una disyunción, $P \vee Q$, siendo P y Q predicados.

5 Una condicional, $P \rightarrow Q$, siendo P y Q predicados.

6 Un bicondicional, $P \leftrightarrow Q$, siendo P y Q predicados.

Lógica de predicados - Sintaxis

↪ Predicados

- ⑦ Una cuantificación universal, $(\forall \alpha \in D.P)$, siendo D el nombre de un dominio de valores y P un predicado.
- ⑧ Una cuantificación existencial, $(\exists \alpha \in D.P)$, siendo D el nombre de un dominio de valores y P un predicado.

Ejercicio

Dada la siguiente declaración de variables:

x, m : entero

v, w : vector[1..N] de enteros

siendo $n \geq 1$, formaliza los siguientes predicados:

- ① v se anula en algún punto.
- ② Todas las componentes de v son positivas.
- ③ x aparece como componente de v .
- ④ El valor de cada componente de v es el doble de su índice

Lógica de predicados - Sintaxis

- Variables en los predicados: **libres y ligadas**

- **Variables libres:** corresponden a las variables del programa.
- **Variables mudas o ligadas:** van asociadas a algún tipo de cuantificador. Ayudan a describir aspectos sobre un dominio.

- Están ligadas al cuantificador más cercano
- Un mismo identificador puede denotar distintas variables

↪ Ejemplo: $\forall x \in D1.(f(x) \wedge (\exists x \in D2.g(x)))$

↪ Ejemplo: $f(x) \wedge (\exists x \in D2.g(x))$

- **Renombramiento de una variable.**

Se puede renombrar de manera consistente las apariciones, ligadas a un mismo cuantificador, de una variable ligada (siempre que la variable por la que se quiere sustituir no sea una variable libre de P)

↪ Ejemplo: El predicado: $\forall x \in D1.(f(x) \wedge (\exists x \in D2.g(x)))$ puede reescribirse:
 $\forall x \in D1.(f(x) \wedge (\exists y \in D2.g(y)))$

NOTA: Para poder renombrar la variable x por la variable y , y **no tiene** que ser una variable libre de P .

Lógica de predicados - Sintaxis

- Variables en los predicados: **libres y ligadas**
 - Para evitar confusiones vamos a nombrar las variables *ligadas* con letras griegas: $\alpha, \beta, \gamma, \dots$

- **Precedencia** de conectivas y cuantificadores
Lista ordenada de mayor a menor prioridad:

$$\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \forall, \exists$$

- $\leftrightarrow$ Los cuantificadores tienen la misma prioridad.
- Ejemplo:

$$P: \forall \alpha \in D1. \forall \beta \in D2. P1 \wedge P2 \vee \neg P3 \rightarrow P4 \leftrightarrow P5 \vee P6$$

$$Q: (\forall \alpha \in D1. (\forall \beta \in D2. (((P1 \wedge P2) \vee (\neg P3)) \rightarrow P4) \leftrightarrow (P5 \vee P6))))$$

- El predicado P es una abreviatura de Q .

Lógica de predicados - Semántica

- El valor de verdad de un **predicado** puede ser *verdadero* o *falso* dependiendo de los valores que tomen las variables libres: $\llbracket P \rrbracket_\sigma$
- Un **estado** σ da valores del tipo adecuado a las variables del programa.

↪ Ejemplo:

$x:\text{natural}, b:\text{boolean}$

$\sigma(x)=6, \sigma(b)=\text{verdadero}$

- En algunos estados el valor de verdad del predicado no está definido:

↪ **Ejemplo: Evaluamos los siguientes predicados atómicos:**

$x, y:\text{natural}, v:\text{vector}[1..N] \text{ de enteros}$

$\sigma(x) = 6 \text{ y } \sigma(y) = 0$

- $\llbracket x \bmod y = 2 \rrbracket_\sigma$: no está bien definido en el estado σ ($\sigma(y) = 0$)
- $\llbracket v[i]=5 \rrbracket_\sigma$: no está bien definido en un estado en el que i no pertenezca al rango $\{1..N\}$
- $\llbracket x > y + 7 \rrbracket_\sigma = F$

Lógica de predicados - Semántica

- $\llbracket P \rrbracket_\sigma = V$ o $\llbracket P \rrbracket_\sigma = F$ significan que un estado P está **bien definido** en el estado σ y que el resultado de su evaluación es V o F .

- La **semántica** de un predicado se define:

$$\llbracket \text{verdadero} \rrbracket_\sigma = V$$

$$\llbracket \text{falso} \rrbracket_\sigma = F$$

$$\llbracket P \rrbracket_\sigma = \text{evaluación de } P \text{ en } \sigma, \text{ si } P \text{ es atómico}$$

$$\hookrightarrow \text{Ejemplo: } \llbracket 2 * x < 10 \rrbracket_\sigma = \text{verdadero si } \sigma(x) = 2$$

$$\llbracket \neg P \rrbracket_\sigma = \begin{cases} \text{verdadero, si } \llbracket P \rrbracket_\sigma = F \\ \text{falso, si } \llbracket P \rrbracket_\sigma = V \end{cases}$$

$$\llbracket P \wedge Q \rrbracket_\sigma = \begin{cases} \text{verdadero, si } \llbracket P \rrbracket_\sigma = V \text{ y } \llbracket Q \rrbracket_\sigma = V \\ \text{falso, e.o.c} \end{cases}$$

$$\llbracket P \vee Q \rrbracket_\sigma = \begin{cases} \text{verdadero, si } \llbracket P \rrbracket_\sigma = V \text{ o } \llbracket Q \rrbracket_\sigma = V \\ \text{falso, e.o.c} \end{cases}$$

$$\llbracket P \rightarrow Q \rrbracket_\sigma = \begin{cases} \text{falso, si } \llbracket P \rrbracket_\sigma = V \text{ y } \llbracket Q \rrbracket_\sigma = F \\ \text{verdadero, si } \llbracket P \rrbracket_\sigma = F \text{ o } \llbracket Q \rrbracket_\sigma = V \end{cases}$$

$$\llbracket P \leftrightarrow Q \rrbracket_\sigma = \begin{cases} \text{verdadero, si } \llbracket P \rrbracket_\sigma = \llbracket Q \rrbracket_\sigma \\ \text{falso, e.o.c.} \end{cases}$$

$$\llbracket \forall \alpha \in D_i. P \rrbracket_\sigma = \begin{cases} \text{verdadero, si para todo } v \in D_i \llbracket P \rrbracket_\sigma[v/\alpha] = V \\ \text{falso, e.o.c.} \end{cases}$$

$$\llbracket \exists \alpha \in D_i. P \rrbracket_\sigma = \begin{cases} \text{verdadero, si existe algún } v \in D_i \text{ tal que } \llbracket P \rrbracket_\sigma[v/\alpha] = V \\ \text{falso, e.o.c.} \end{cases}$$

Lógica de predicados

↪ Ejemplos:

- $x: \text{nat}, b: \text{bool}, v: \text{vector}[1..100] \text{ de ent}$

- Si $\sigma(x) = 2, \sigma(b) = V$ entonces:

↪ $\llbracket b \wedge (3 * x < 10 \vee (\exists \alpha \in [1..100]. v[\alpha] = 0)) \rrbracket_{\sigma} = V$

- Si $\sigma(x) = 10, \sigma(b) = V, \sigma(v) = [0, 1, 2, \dots, 99]$ entonces:

↪ $\llbracket b \wedge (3 * x < 10 \vee (\exists \alpha \in [1..100]. v[\alpha] = 0)) \rrbracket_{\sigma} = V$

- Si $\sigma(b) = F$ entonces:

↪ $\llbracket b \wedge (3 * x < 10 \vee (\exists \alpha \in [1..100]. v[\alpha] = 0)) \rrbracket_{\sigma} = F$

↪ Definiciones:

- Un estado **satisface** un predicado cuando lo hace cierto.

- $\llbracket P \rrbracket_{\sigma} = v$

- Un predicado P es **válido** cuando cualquier estado lo satisface.

- Diremos que dos predicados P y Q son **equivalentes**, denotado por $P \equiv Q$, si satisfacen el mismo conjunto de estados:

$$\forall \sigma \in \epsilon. \llbracket P \rrbracket_{\sigma} = \llbracket Q \rrbracket_{\sigma}$$

Lógica de predicados - Leyes de equivalencia

Leyes conmutativas, asociativas y de idempotencia de $\wedge$ y $\vee$

- $P \wedge Q \Leftrightarrow Q \wedge P$
- $P \vee Q \Leftrightarrow Q \vee P$
- $P \wedge (Q \wedge R) \Leftrightarrow (P \wedge Q) \wedge R$
- $P \vee (Q \vee R) \Leftrightarrow (P \vee Q) \vee R$
- $P \wedge P \Leftrightarrow P$
- $P \vee P \Leftrightarrow P$

Leyes distributivas, de absorción y de elemento neutro

- $P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$
- $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$
- $P \wedge (P \vee Q) \Leftrightarrow P$
- $P \vee (P \wedge Q) \Leftrightarrow P$
- $P \wedge \text{falso} \Leftrightarrow \text{falso}$
- $P \vee \text{cierto} \Leftrightarrow \text{cierto}$
- $P \wedge \text{cierto} \Leftrightarrow P$
- $P \vee \text{falso} \Leftrightarrow P$

Lógica de predicados - Leyes de equivalencia

Leyes de la negación

- $\neg\neg P \Leftrightarrow P$
- $P \vee \neg P \Leftrightarrow \text{cierto}$
- $P \wedge \neg P \Leftrightarrow \text{falso}$

Leyes de De Morgan

- $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$
- $\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$

Leyes de $\rightarrow$ y $\leftrightarrow$

- $P \rightarrow Q \Leftrightarrow \neg P \vee Q$
- $P \leftrightarrow Q \Leftrightarrow (P \rightarrow Q) \wedge (Q \rightarrow P)$

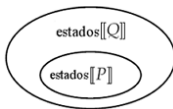
Leyes de $\forall$ y $\exists$

- $\neg(\forall \alpha \in D. P) \equiv (\exists \alpha \in D. \neg P)$
- $\neg(\exists \alpha \in D. P) \equiv (\forall \alpha \in D. \neg P)$

Especificación con predicados

Especificación con predicados

- Un predicado P puede definirse a partir del *conjunto de estados* que lo satisfacen: $\text{estados}[[P]]$
 $\hookrightarrow$ Ejemplo:
 x, y : *enteros*
 $P \equiv (x \text{ div } y > 1)$
 $\text{estados}[[P]] = (2, 1), (-2, -1), (5, 2), (-7, -3), \dots$
- Cuanto mayor sea el conjunto de estados definidos por un predicado *más débil* es el predicado.
 $\hookrightarrow$ Relación de orden parcial entre los predicados, "ser más fuerte que" (o "más débil que")
- Un predicado P es *más fuerte* que Q si el conjunto de estados que satisfacen P está incluido en el conjunto de estados que satisfacen Q . Es decir, siempre que se cumpla P se cumple Q . $\text{estados}[[P]] \subseteq \text{estados}[[Q]]$.
 - P es *más fuerte* que Q , $P \Rightarrow Q$.
 - Q es *más débil* que P , $\text{estados}[[P]] \subseteq \text{estados}[[Q]]$, $Q \Leftarrow P$.



Especificación con predicados

$\hookrightarrow$ Por ejemplo, $x \geq 2 \Rightarrow x \geq 1$ y $x > 0 \Rightarrow x \geq 0$

Ejercicio

Emplear la notación gráfica $P \rightarrow Q$ para indicar que " P es más fuerte que Q " para el siguiente conjunto de predicados:

- 1 $P_1 \equiv x > 0$
- 2 $P_2 \equiv (x > 0) \wedge (y > 0)$
- 3 $P_3 \equiv (x > 0) \vee (y > 0)$
- 4 $P_4 \equiv y \geq 0$
- 5 $P_5 \equiv (x \geq 0) \wedge (y \geq 0)$

Especificación con predicados

- Ver los conjuntos que definen $\Rightarrow$ por lo general no son calculables por ser de cardinalidad infinita
- Necesitamos establecer comparaciones entre las restricciones que imponen los predicados buscando subexpresiones implicadas o que implican a otras subexpresiones (o la las expresiones completas) de los predicados.
- Patrones para buscar subexpresiones implicadas por otras subexpresiones:
Para cuales quiera subexpresiones Q, R :
 - $Q \Rightarrow Q \vee R$: es menos restrictiva una disyunción que cualquiera de sus disyuntantes
 - $Q \wedge R \Rightarrow Q$: es más restrictiva una conjunción que cualquiera de sus conjuntantes

Especificación con predicados

- Hablaremos de **fortalecer** un predicado P cuando pasemos de P a un predicado P' tal que $P' \Rightarrow P$

Por ejemplo, $P' \equiv P \wedge Q$, ya que:

$$\text{estados}[[P \wedge Q]] = \text{estados}[[P]] \cap \text{estados}[[Q]] \subseteq \text{estados}[[P]]$$

y por tanto, $P \wedge Q \Rightarrow P$.

- Hablaremos de **debilitar** un predicado P cuando pasemos de P a un predicado P' tal que $P \Rightarrow P'$.

Por ejemplo, $P' \equiv P \vee Q$, ya que:

$$\text{estados}[[P \vee Q]] = \text{estados}[[P]] \cup \text{estados}[[Q]] \supseteq \text{estados}[[P]]$$

y por tanto, $P \Rightarrow P \vee Q$.

- El predicado **más fuerte** que cualquier otro predicado es *falso*, ya que:

$$\text{falso} \wedge P \equiv \text{falso}, \text{ para todo } P.$$

- El predicado **más débil** que cualquier otro predicado es *cierto*, ya que:

$$P \vee \text{cierto} \equiv \text{cierto}, \text{ para todo } P.$$

Especificación con predicados

- Otra forma de debilitar un predicado: sustituir una constante en P por una variable en P' acotada en límites (mismo tipo, donde la constante sea uno de los valores del dominio).
 P es un caso particular de P' .

Especificación con predicados - Convenios sobre cuantificadores

- Cuantificadores

- Universal $\forall$
- Existencial $\exists$
- Sumatorio $\sum$
- Producto $\prod$
- Conteo N

- En los predicados donde aparecen cuantificadores se utilizan **expresiones de la forma $\{a..b\}$** .
- Convenimos designar el siguiente conjunto:

$$\{a..b\} = \begin{cases} \emptyset & , \text{ si } a > b \\ \{a\} \cup \{a+1..b\} & , \text{ si } a \leq b \end{cases}$$

Especificación con predicados - Convenios sobre cuantificadores

$$\forall \alpha \in \{a..b\}.P(\alpha) = \begin{cases} \text{cierto} & , \text{ si } a > b \\ f(a) \wedge (\forall \alpha \in \{a+1..b\}.P(\alpha)) & , \text{ si } a \leq b \end{cases}$$

$$\exists \alpha \in \{a..b\}.P(\alpha) = \begin{cases} \text{falso} & , \text{ si } a > b \\ f(a) \vee (\exists \alpha \in \{a+1..b\}.P(\alpha)) & , \text{ si } a \leq b \end{cases}$$

$$\sum \alpha \in \{a..b\}.P(\alpha) = \begin{cases} 0 & , \text{ si } a > b \\ E(a) + (\sum \alpha \in \{a+1..b\}.P(\alpha)) & , \text{ si } a \leq b \end{cases}$$

$$\prod \alpha \in \{a..b\}.P(\alpha) = \begin{cases} 1 & , \text{ si } a > b \\ E(a) * (\prod \alpha \in \{a+1..b\}.P(\alpha)) & , \text{ si } a \leq b \end{cases}$$

$$N \alpha \in \{a..b\}.P(\alpha) = \begin{cases} 0 & , \text{ si } a > b \\ f(a) + (N \alpha \in \{a+1..b\}.P(\alpha)) & , \text{ si } a \leq b \end{cases}$$

↪ **Nota:** Las definiciones de variables **libres** y **ligadas** se extienden igualmente a los cuantificadores matemáticos.

Convenios sobre cuantificadores - Ejemplos

↪ $\forall$: Si $n=0$, tendríamos:

$$(\forall \alpha \in \{1..n\}.a[\alpha] \neq x) \equiv \text{cierto}$$

- Un predicado cuantificado con $\forall$ puede entenderse como un $\wedge$ generalizado:

$$(\forall \alpha \in \{1..n\}.a[\alpha] \neq x) \equiv (a[1] \neq x \wedge a[2] \neq x \wedge \dots \wedge a[n] \neq x)$$

↪ **cierto** es el **elemento neutro** de la conectiva $\wedge$

↪ $\exists$: Si $n=0$, tendríamos:

$$(\exists \alpha \in \{1..n\}.a[\alpha] \neq x) \equiv \text{falso}$$

- Un predicado cuantificado con $\exists$ puede entenderse como un $\vee$ generalizado:

$$(\exists \alpha \in \{1..n\}.a[\alpha] \neq x) \equiv (a[1] \neq x \vee a[2] \neq x \vee \dots \vee a[n] \neq x)$$

↪ **falso** es el **elemento neutro** de la conectiva $\vee$

↪ N : $f(a) + (N\alpha \in \{a+1..b\}.P(\alpha))$, si $a \leq b$

- $f(a)=1$ si $\llbracket P \rrbracket = V$
- $f(a)=0$ si $\llbracket P \rrbracket = F$

Especificación con predicados - Convenios sobre sustituciones

- Denotamos por P_x^E al predicado que resulta de **sustituir** en P todas las apariciones **libres** de la variable x por la expresión E .
- En general, se denota por $P_{x_1, \dots, x_n}^{E_1, \dots, E_n}$ al predicado resultante de sustituir en P , simultáneamente, las apariciones de $x_1, \dots, x_n$ por las expresiones $E_1, \dots, E_n$
- **Condiciones**
 - La expresión E no ha de nombrar variables ligadas de P .
 - La expresión E ha de ser del mismo tipo de datos que x .

Especificación con predicados - Convenios sobre sustituciones

↪ Ejemplos:

- $P \equiv \exists \alpha \in \{1..n\}. v[\alpha] = x. P_n^{n+1}$

El resultado de la sustitución es: $\exists \alpha \in \{1..n+1\}. v[\alpha] = x$

- $P \equiv \forall \alpha \in \{1..n\}. v[\alpha] = x. P_\alpha^{\alpha+1}$

La variable está ligada luego no se sustituye.

- $P \equiv \forall i \in \{1..n\}. v[i] < x. P_x^{i+1}$

- No es correcto: $(\forall i \in \{1..n\}. v[i] < i + 1)$. La variable i es una variable ligada.

- Renombramos la variable ligada i y luego hacemos la sustitución.

$$\forall \alpha \in \{1..n\}. v[\alpha] < x$$

- Sustituimos: $\forall \alpha \in \{1..n\}. v[\alpha] < i + 1$

- $P \equiv \forall \alpha \in \{1..n\}. v[\alpha] = x * y. P_{x,y}^{i,j}$

El resultado de la sustitución es: $(\forall \alpha \in \{1..n\}. v[\alpha] = i * j)$

Convenios sobre sustituciones - Ejercicio

Ejercicio

Si $P \equiv \forall \alpha \in \{1..n\}. v[\alpha] = x + 2y$, realiza las siguientes sustituciones:

① p_x^{y+1}

② $p_{x,y}^{y,x}$

③ p_x^{x+1}

④ $p_\alpha^{\alpha+1}$

Especificación pre/post

Especificación formal pre/post

- Técnica basada en el uso de la lógica de predicados
- Considera el algoritmo S como una caja negra del que se conoce solamente su interfaz.
- S Actúa como una función de estados en estados: comienza su ejecución en un estado **inicial** válido, descrito por el valor de los parámetros de entrada, y termina en un estado **final** en el que los parámetros de salida contienen los resultados esperados.



Especificación formal pre/post

- La especificación formal de un algoritmo constará de los siguientes elementos:

$$\{Q\}S\{R\}$$

donde:

- Precondición (Q): Describe las obligaciones del usuario. Sus variables libres son los parámetros de entrada o entrada/salida de S (no los de salida)
- S : función a especificar.
- Postcondición (R): Describe las obligaciones del implementador. Sus variables libres son los parámetros de cualquier clase de S .

Especificación formal pre/post

- S: **Nomenclatura** para la declaración de la función:

fun nombre($p_1:t_1; \dots; p_n:t_n$) dev ($q_1:r_1; \dots; q_n:r_n$)

donde:

- p_i : son variables de entrada (parámetros de entrada). El algoritmo no lo puede modificar.
- q_i : son variables de salida (parámetros de salida). Contienen el resultado y su valor inicial es irrelevante.
- t_i y r_i : tipos de datos.
- Las únicas variables libres que se permiten en los predicados son sus parámetros.
- **Parámetros de entrada y salida**: la variable tiene un valor inicial relevante para el algoritmo y devuelve el resultado (ejemplo: ordenación de un vector).

Especificación formal pre/post

- S : **Nomenclatura** para la declaración de un procedimiento:

accion nombre($p_1:\mathbf{cf} \ t_1; \dots; p_n:\mathbf{cf} \ t_n$)

donde:

- $\mathbf{cf}$ puede ser:
 - *ent o nada*: parámetro de sólo entrada
 - *sal*: parámetro de sólo salida
 - *ent/sal*: parámetro de entrada/salida

$\hookrightarrow \{Q\}S\{R\}$ se lee:

- Si el algoritmo S comienza su ejecución en un estado descrito por Q , S termina, y lo hace en un estado descrito por R .
- No se dice nada sobre qué sucederá si el algoritmo se ejecuta en un estado que no satisface la precondition.

Especificación formal pre/post

- La especificación de un algoritmo tiene doble destinatario:
 - Los **usuarios** del algoritmo. La especificación debe recoger todo lo necesario para un uso correcto del mismo.
 - El **implementador** del algoritmo. La especificación debe definir los requisitos que cualquier implementación válida debe satisfacer.

	Precondición	Postcondición
Usuario	Debe garantizar que se cumpla inicialmente	Tiene derecho a esperar que S termine en un estado que la cumpla
Implementador	Tiene derecho a esperar que se cumpla	Debe garantizar que S termine en un estado que la cumpla

Especificación formal pre/post - Ejemplos

Ejemplo

Especifica una función que dado un vector de 1000 elementos enteros y un número entero n , siendo $0 \leq n \leq 1000$, devuelva un booleano que indique si el valor de alguno de los elementos del vector es igual a la suma de todos los elementos que le preceden en el mismo.

En el caso en que el primer elemento del vector sea 0 la función deberá devolver *falso*.
En el caso en el el primer elemento del vector sea 0 y n mayor que 0, la función deberá devolver *verdadero*.

Especificación formal pre/post - Ejemplos

$\{Q \equiv 0 \leq n \leq 1000\}$

fun esSuma(v: vector[1..100] de enteros; n: entero) dev (b: booleano);

$\{R \equiv b = \exists \alpha \in \{1..n\}. v[\alpha] = \sum \beta \in \{1..\alpha - 1\} v[\beta]\}$

- ¿serían admisibles lladas a "esSuma" con valores de n negativos o mayores que 1000?
- ¿Y con $n=0$? $\Rightarrow$ ¿falso?
- ¿Y si $v[1] = 0$ y $n \geq 1$?